SRMECOTECH

Database Schema Specification

PostgreSQL database gps_tracking (Flyway V1 to V8), with the Redis key space, Kafka topics and state stores of the GPS Processing Layer

Document identifier	SRM-GPS-DBS-001
Version	0.9
Status	Draft for review (not yet baselined)
Date	29 October 2026
Prepared by	Sanjoy Chattopadhay, SRMECOTECH
Classification	Internal
Governing standard	ISO/IEC/IEEE 29148:2018 (data requirements)

Companion to SRM-GPS-SRS-001. The schema described here is the one created by the Flyway migrations V1 to V8 in gps-application/src/main/resources/db/migration; if the two ever differ, the migrations are authoritative and this document shall be corrected.

Document Control

This document is placed under configuration control. Changes are made only through the change-control procedure in the companion document “SRS Rules and Regulations” and are recorded in the revision history below. A version number ending in “.0” denotes a baselined (approved) issue; any other number denotes a working draft.

Table 1: *Revision history*

Version	Date	Author	Description of change
0.1	19 Oct 2026	S. Chattopadhay	Tables listed in README section 11 (V1 to V4).
0.9	29 Oct 2026	S. Chattopadhay	First formal specification: data dictionary for all 15 tables (V1 to V8), constraints, indexes, relationships, lifecycle, Redis, Kafka and state stores, full DDL.

Table 2: *Review and approval*

Role	Name	Signature	Date
Technical lead / architect			
Database administrator			
Quality assurance lead			

Table of Contents

Document Control 2
1 Introduction 5
1.1 Purpose 5
1.2 Scope 5
1.3 References 5
1.4 Notation 5
2 Database Overview 6
2.1 Subject Areas 6
3 Design Principles and Naming Conventions 8
3.1 Design Principles 8
3.2 Naming Conventions 8
4 Data Dictionary 9
4.1 Table tenant 9
4.2 Table vehicle 9
4.3 Table device 10
4.4 Table device_assignment 10
4.5 Table gps_position 11
4.6 Table vehicle_motion_state 12
4.7 Table gps_trip 13
4.8 Table gps_stop 14
4.9 Table gps_trip_late_reading 15
4.10 Table vehicle_event 16
4.11 Table dead_letter 17
4.12 Table rejected_frame 18
4.13 Table connection_setting 19
4.14 Table vehicle_day_summary 19
4.15 Table report_setting 20
5 Relationships and Referential Integrity 22
6 Value Sets 23
7 Index Strategy and Main Queries 24
8 Data Integrity, Idempotency and Concurrency 25
9 Data Lifecycle: Retention, Clearing, Backup and Recovery 26
9.1 Retention 26
9.2 Clear Database and Demo Reset 26
9.3 Backup and Recovery 26

9.4 Capacity Estimate....26
10 Migration History and Change Rules....28
11 Non-Relational Data Stores....29
11.1 Redis Key Space....29
11.2 Kafka Topics....29
11.3 Kafka Streams State Stores....30
12 External Source: smart_truck (Read-Only)....31
13 Open Issues....32
14 Appendices....33
14.1 Appendix A: Complete DDL....33
14.1.1 V1__create_master_data_tables.sql....33
14.1.2 V2__create_gps_position_table.sql....33
14.1.3 V3__create_vehicle_motion_state_table.sql....34
14.1.4 V4__create_gps_trip_and_gps_stop_tables.sql....34
14.1.5 V5__distance_details_and_vehicle_events.sql....35
14.1.6 V6__dead_letter_and_rejected_frame_archive.sql....36
14.1.7 V7__connection_setting.sql....36
14.1.8 V8__vehicle_day_summary_and_report_setting.sql....37

1 Introduction

1.1 Purpose

This document specifies the persistent data of the GPS Processing Layer: the PostgreSQL database gps_tracking in full, and the Redis key space, Kafka topics and Kafka Streams state stores that hold the rest of the platform's state. It is the physical counterpart of Sections 3.2 and 6 of the Software Requirements Specification SRM-GPS-SRS-001 and satisfies its data requirements DR-001 to DR-010.

1.2 Scope

- Fifteen PostgreSQL tables created by Flyway migrations V1 to V8: master data, telemetry, trip detection, events, reporting, archives and configuration.
- Constraints, indexes, relationships, value sets, write patterns, idempotency keys, retention and recovery.
- Redis keys (live state, configuration, counters), Kafka topics and Kafka Streams state stores.
- The read-only legacy source smart_truck, as far as this platform reads it.

1.3 References

Table 3: *Referenced documents*

Ref.	Document
[S1]	SRM-GPS-SRS-001, Software Requirements Specification: GPS Processing Layer.
[S2]	SRM-GPS-SRS-RUL-001, SRS Rules and Regulations.
[S3]	gps-application/src/main/resources/db/migration/V1 to V8 (authoritative DDL).
[S4]	README.md sections 11, 12, 14 and 15; docs/DATA_SAFETY_GUIDE.md; docs/REPORTS_GUIDE.md.
[S5]	PostgreSQL 14 documentation: data types, constraints (EXCLUDE with btree_gist), indexes.

1.4 Notation

Table 4: *Notation used in this document*

Symbol	Meaning
PK	Primary key
UK	Unique key (UNIQUE constraint)
FK	Foreign key
CK	CHECK constraint
EX	Exclusion constraint (EXCLUDE USING gist)
IX	Secondary index
Null = N	Column is NOT NULL
Logical reference	A column that names a row of another table (for example imei) without a foreign key; integrity is kept by the application (Section 5).
timestamptz	timestamp with time zone; every value is stored and compared as an absolute instant (UTC).

2 Database Overview

Table 5: *Database facts*

Item	Value
DBMS	PostgreSQL 14 (Docker image postgres:14; native 14.22)
Database / schema	gps_tracking / public
Application user	gps_app (password from local-secrets.properties or .env; never in version control)
Extensions	btree_gist (needed for the exclusion constraint on device_assignment)
Schema management	Flyway, migrations V1 to V8; Hibernate ddl-auto = validate (it never changes the schema)
Time zone	JDBC session time zone UTC (hibernate.jdbc.time_zone); all timestamps are timestamptz
Character set	UTF-8
Connection	jdbc:postgresql://localhost:5432/gps_tracking (Docker: postgres:5432, host port 5433)

2.1 Subject Areas

Table 6: *Tables by subject area*

Subject area	Tables	Written by
Master data	tenant, vehicle, device, device_assignment	SimulationMasterDataSeeder (future: master-data admin API)
Telemetry	gps_position	Consumer group gps-position-storage-service (or the direct flow)
Trip detection	vehicle_motion_state, gps_trip, gps_stop, gps_trip_late_reading	Consumer group gps-trip-service
Events	vehicle_event	Speed-violation, vehicle-state, stoppage and ignition consumers
Reporting	vehicle_day_summary, report_setting	DaySummaryJob; Reports → Cost assumptions
Archives	dead_letter, rejected_frame	gps-dead-letter-archive; gps-rejected-frame-logger
Configuration	connection_setting	Developer → Connections

Figure 1 (next page) shows all tables and their relationships.

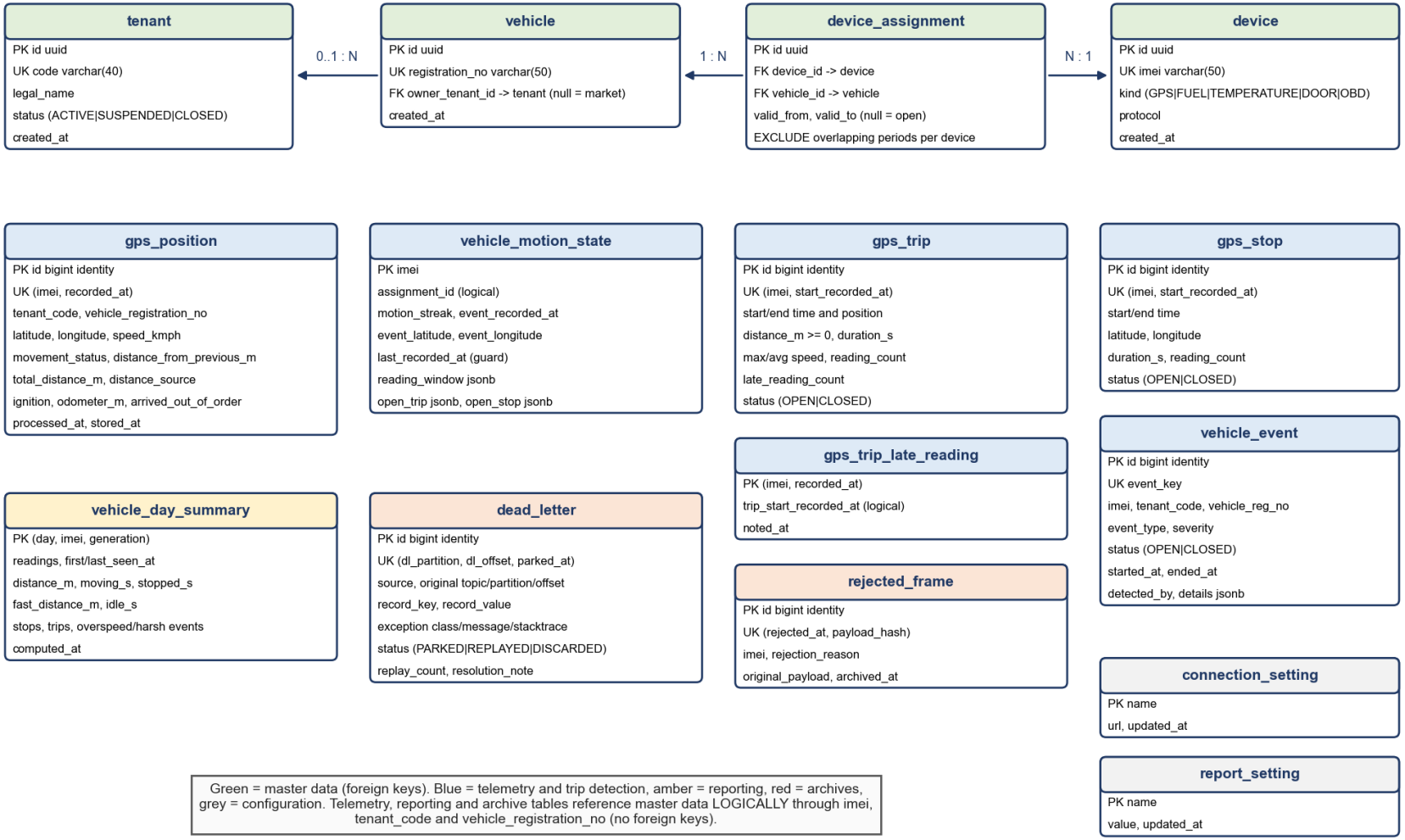


Figure 1: *Entity-relationship overview of gps_tracking*

3 Design Principles and Naming Conventions

3.1 Design Principles

Table 7: *Design decisions*

Decision	Rationale
Master data uses uuid keys and foreign keys.	Few rows, changed rarely, referenced by other master data: the database guarantees integrity.
Telemetry uses bigint identity keys and logical references (imei, tenant_code, vehicle_registration_no).	Written at high rate; readable without joins; must survive master-data changes; planned to move to a time-series store.
Device assignments are periods, never a “current vehicle” column.	A late reading must be matched to the vehicle the device was in at the reading's time (BR-004).
Every consumer-written table has a natural idempotency key.	Kafka delivers at least once; a redelivered record must not create a second row (Section 8).
Trip state is updated with a guard on last_recorded_at.	A repeated or late reading updates 0 rows, so the whole transaction is rolled back.
Value sets are CHECK constraints, not lookup tables.	Small, stable sets; the check documents the allowed values next to the column.
Archives are never deleted; status changes instead.	Every record that did not become a position stays traceable (BR-006).
Reports read a summary table.	One row per vehicle per day keeps month reports small whatever the position volume.

3.2 Naming Conventions

- Tables: singular, lower snake_case (gps_position, not GpsPositions).
- Time columns end in `_at` and are timestamptz; a date-only column is named `day`.
- Units are suffixes: `_m` metres, `_s` seconds, `_kmph` km/h, `_meters` in older columns of the same meaning.
- Constraints: <table>_<rule> (gps_trip_distance_not_negative); indexes: <table>_<columns or purpose>.
- Status columns hold upper-case words (OPEN, CLOSED, PARKED …).

4 Data Dictionary

For each table: purpose, writer and readers, write pattern, columns, constraints and indexes. Column order follows the migration that created it; columns added later (V5) are listed at the end.

4.1 Table tenant

Item	Value
Purpose	A customer company on the platform.
Created in	V1__create_master_data_tables.sql
Written by / read by	SimulationMasterDataSeeder / DeviceVehicleDirectory, reports filters
Expected size	Tens of rows

Table 8: *tenant columns*

#	Column	Type	Null	Default	Description
1	**id**	uuid	N		Surrogate key, generated by the application.
2	**code**	varchar(40)	N		Short lower-case code stamped on every position, never re-used (BR-003).
3	**legal_name**	varchar(255)	N		Registered company name.
4	**status**	varchar(20)	N	'ACTIVE'	ACTIVE, SUSPENDED or CLOSED.
5	**created_at**	timestamptz	N	now()	Creation time.

Table 9: *tenant constraints*

Constraint	Kind	Definition and purpose
tenant_pkey	PK	(id)
tenant_code_key	UK	(code): one code per tenant, platform-wide.
tenant_code_format	CK	code ~ '^[a-z0-9][a-z0-9_-]*$'
tenant_status_values	CK	status IN ('ACTIVE','SUSPENDED','CLOSED')

4.2 Table vehicle

Item	Value
Purpose	A truck, identified by its registration number.
Created in	V1
Written by / read by	SimulationMasterDataSeeder / DeviceVehicleDirectory
Expected size	Hundreds to thousands of rows

Table 10: *vehicle columns*

#	Column	Type	Null	Default	Description
1	**id**	uuid	N		Surrogate key.
2	**registration_no**	varchar(50)	N		Registration plate, unique across the platform (BR-002).
3	**owner_tenant_id**	uuid	Y		Owning tenant; null means a market vehicle.
4	**created_at**	timestamptz	N	now()	Creation time.

Table 11: vehicle constraints

Constraint	Kind	Definition and purpose
vehicle_pkey	PK	(id)
vehicle_registration_no_key	UK	(registration_no)
vehicle_owner_tenant_id_fkey	FK	owner_tenant_id → tenant(id); default NO ACTION on delete and update.

4.3 Table device

Item	Value
Purpose	A tracking device, known only by its IMEI.
Created in	V1
Written by / read by	SimulationMasterDataSeeder / DeviceVehicleDirectory
Expected size	Hundreds to thousands of rows

Table 12: device columns

#	Column	Type	Null	Default	Description
1	**id**	uuid	N		Surrogate key.
2	**imei**	varchar(50)	N		Device identity; the Kafka record key of all its messages.
3	**kind**	varchar(20)	N	'GPS'	GPS, FUEL, TEMPERATURE, DOOR or OBD.
4	**protocol**	varchar(50)	Y		Device protocol name, for the future parser.
5	**created_at**	timestamptz	N	now()	Creation time.

Table 13: device constraints

Constraint	Kind	Definition and purpose
device_pkey	PK	(id)
device_imei_key	UK	(imei)
device_kind_values	CK	kind IN ('GPS','FUEL','TEMPERATURE','DOOR','OBD')

4.4 Table device_assignment

Item	Value
Purpose	The period during which a device is installed in a vehicle.
Created in	V1
Written by / read by	SimulationMasterDataSeeder / DeviceVehicleDirectory (cached per period)
Rule	A device is in at most one vehicle at any instant (BR-001). The period is half-open: [valid_from, valid_to).

Table 14: device_assignment columns

#	Column	Type	Null	Default	Description
1	**id**	uuid	N		Surrogate key; also stored in vehicle_motion_state.assignment_id.

#	Column	Type	Null	Default	Description
2	**device_id**	uuid	N		The device.
3	**vehicle_id**	uuid	N		The vehicle.
4	**valid_from**	timestamptz	N		Start of the installation (inclusive).
5	**valid_to**	timestamptz	Y		End (exclusive); null while still installed.

Table 15: *device_assignment constraints*

Constraint	Kind	Definition and purpose
device_assignment_pkey	PK	(id)
device_assignment_device_id_fkey	FK	device_id → device(id)
device_assignment_vehicle_id_fkey	FK	vehicle_id → vehicle(id)
device_assignment_period_is_valid	CK	valid_to IS NULL OR valid_to > valid_from
device_assignment_one_vehicle_per_device_at_a_time	EX	EXCLUDE USING gist (device_id WITH =, tstzrange(valid_from, valid_to) WITH &&): no two periods of one device overlap.

Table 16: *device_assignment indexes*

Index	Columns	Serves
device_assignment_vehicle	(vehicle_id)	Devices of a vehicle
(exclusion constraint index)	gist (device_id, tstzrange)	Assignment of a device at an instant

4.5 Table gps_position

Item	Value
Purpose	Every accepted reading, after normalisation, with tenant and vehicle resolved at the reading's time.
Created in	V2 (columns total_distance_m, distance_source, ignition, odometer_m added in V5)
Written by / read by	gps-position-storage-service / position APIs, report summary job, vehicle-day sheet, previous-reading fallback
Write pattern	Insert only; a duplicate (imei, recorded_at) is skipped (checked first, and enforced by the unique key).
Expected volume	About 216,000 rows a day for 150 vehicles at one reading a minute; about 6.5 million a day in compressed simulator timing.

Table 17: *gps_position columns*

#	Column	Type	Null	Default	Description
1	**id**	bigint identity	N	generated	Surrogate key.
2	**imei**	varchar(50)	N		Device (logical reference to device.imei).
3	**tenant_code**	varchar(40)	Y		Tenant at the reading's time; null when the device was unassigned.
4	**vehicle_registration_no**	varchar(50)	Y		Vehicle at the reading's time; null when unassigned.
5	**recorded_at**	timestamptz	N		Time the device recorded the reading.
6	**latitude**	double precision	N		WGS-84 degrees, −90 to 90.
7	**longitude**	double precision	N		WGS-84 degrees, −180 to 180.

#	Column	Type	Null	Default	Description
8	**speed_kmph**	integer	N		Reported speed, 0 to 150.
9	**movement_status**	varchar(10)	N		MOVING or STOPPED.
10	**distance_from_previous_m**	double precision	N		Counted distance since the previous in-order reading (0 for first, jitter, jump and late readings).
11	**seconds_since_previous**	bigint	N		Seconds since the previous in-order reading (0 for first and late readings).
12	**arrived_out_of_order**	boolean	N		True for a late reading.
13	**processed_at**	timestamptz	N		When the normaliser produced the position.
14	**stored_at**	timestamptz	N	now()	When the row was inserted.
15	**total_distance_m**	double precision	Y		Running distance total of the device (V5).
16	**distance_source**	varchar(20)	Y		FIRST_READING, GPS, DEVICE_ODOMETER, JITTER_FILTERED, JUMP_FILTERED or OUT_OF_ORDER (V5).
17	**ignition**	boolean	Y		Device ignition input, when wired (V5).
18	**odometer_m**	bigint	Y		Device odometer, when reported (V5).

Table 18: *gps_position constraints*

Constraint	Kind	Definition and purpose
gps_position_pkey	PK	(id)
gps_position_one_reading_per_device_per_moment	UK	(imei, recorded_at): Kafka redelivery stores a reading once (BR-005).
gps_position_latitude_range	CK	latitude BETWEEN -90 AND 90
gps_position_longitude_range	CK	longitude BETWEEN -180 AND 180
gps_position_movement_status_values	CK	movement_status IN ('MOVING','STOPPED')

Table 19: *gps_position indexes*

Index	Columns	Serves
gps_position_vehicle_time	(vehicle_registration_no, recorded_at DESC)	Latest position and history of a vehicle
gps_position_stored_at	(stored_at DESC)	Recently stored positions
gps_position_recorded_at	(recorded_at)	Report job: one day of positions for all vehicles (V8)
(unique key index)	(imei, recorded_at)	Duplicate check; previous reading of a device after restart

4.6 Table vehicle_motion_state

Item	Value
Purpose	Working memory of the trip detector: one row per device.
Created in	V3
Written by / read by	gps-trip-service (TripDetectionService, VehicleMotionStateStore) / vehicle inspector
Write pattern	Upsert in the same transaction as the gps_trip and gps_stop rows it produces; guarded by last_recorded_at < the new reading's time.

Table 20: vehicle_motion_state columns

#	Column	Type	Null	Default	Description
1	**imei**	varchar(50)	N		Device (PK).
2	**assignment_id**	uuid	Y		device_assignment of the newest reading; a change closes the open trip. Null: not in a vehicle.
3	**tenant_code**	varchar(40)	Y		Tenant of the newest reading.
4	**vehicle_registration_no**	varchar(50)	Y		Vehicle of the newest reading.
5	**motion_streak**	boolean	N		True between “started moving” and “stopped” (moving state).
6	**event_recorded_at**	timestamptz	N		When the vehicle last started moving or stopped.
7	**event_latitude**	double precision	N		Where that happened.
8	**event_longitude**	double precision	N		Where that happened.
9	**last_recorded_at**	timestamptz	N		Newest reading applied; older or equal readings are skipped.
10	**reading_window**	jsonb	N		In-order readings of the last min-duration (a few dozen small objects), oldest first.
11	**open_trip**	jsonb	Y		The trip in progress (same values as its OPEN row in gps_trip), or null.
12	**open_stop**	jsonb	Y		The stop in progress (same values as its OPEN row in gps_stop), or null.
13	**updated_at**	timestamptz	N	now()	Last update.

Table 21: vehicle_motion_state constraints

Constraint	Kind	Definition and purpose
vehicle_motion_state_pkey	PK	(imei)

4.7 Table gps_trip

Item	Value
Purpose	A trip: from the moment a vehicle starts moving until it stops.
Created in	V4
Written by / read by	gps-trip-service / GET /api/trips, report summary job, vehicle-day sheet
Write pattern	Upsert on (imei, start_recorded_at): an OPEN trip is extended with every reading and finally CLOSED.

Table 22: gps_trip columns

#	Column	Type	Null	Default	Description
1	**id**	bigint identity	N	generated	Surrogate key.
2	**imei**	varchar(50)	N		Device.
3	**tenant_code**	varchar(40)	Y		Tenant from the assignment valid at the trip's readings.
4	**vehicle_registration_no**	varchar(50)	Y		Vehicle from the same assignment.
5	**start_recorded_at**	timestamptz	N		Start, back-dated to the first movement.

#	Column	Type	Null	Default	Description
6	**end_recorded_at**	timestamptz	N		End (moves with every reading while OPEN).
7	**start_latitude / start_longitude**	double precision	N		Start position.
8	**end_latitude / end_longitude**	double precision	N		End position.
9	**distance_m**	double precision	N		Sum of distance_from_previous_m of in-order readings; never end − start odometer.
10	**duration_s**	bigint	N		end − start in seconds.
11	**max_speed_kmph**	integer	N		Highest reported speed.
12	**avg_speed_kmph**	double precision	N		distance ÷ duration.
13	**reading_count**	integer	N		In-order readings in the trip.
14	**late_reading_count**	integer	N	0	Late readings that fell into the trip (not in distance or duration).
15	**status**	varchar(10)	N		OPEN or CLOSED.
16	**updated_at**	timestamptz	N	now()	Last update.

Table 23: *gps_trip constraints*

Constraint	Kind	Definition and purpose
gps_trip_pkey	PK	(id)
gps_trip_one_trip_per_device_per_start	UK	(imei, start_recorded_at): a replayed reading updates the same trip.
gps_trip_status_values	CK	status IN ('OPEN','CLOSED')
gps_trip_ends_after_it_starts	CK	end_recorded_at >= start_recorded_at
gps_trip_distance_not_negative	CK	distance_m >= 0

Table 24: *gps_trip indexes*

Index	Columns	Serves
gps_trip_vehicle_time	(vehicle_registration_no, start_recorded_at DESC)	Trips of a vehicle in a period

4.8 Table gps_stop

Item	Value
Purpose	A stop: the vehicle stays within the minimum distance for at least the minimum duration.
Created in	V4
Written by / read by	gps-trip-service / GET /api/stops, report summary job, vehicle-day sheet
Write pattern	Upsert on (imei, start_recorded_at).

Table 25: *gps_stop columns*

#	Column	Type	Null	Default	Description
1	**id**	bigint identity	N	generated	Surrogate key.
2	**imei**	varchar(50)	N		Device.
3	**tenant_code**	varchar(40)	Y		Tenant at the stop.

#	Column	Type	Null	Default	Description
4	**vehicle_registration_no**	varchar(50)	Y		Vehicle at the stop.
5	**start_recorded_at**	timestamptz	N		Start of the stop.
6	**end_recorded_at**	timestamptz	N		End (moves while OPEN).
7	**latitude / longitude**	double precision	N		Stop position.
8	**duration_s**	bigint	N		end − start in seconds.
9	**reading_count**	integer	N		Readings during the stop.
10	**status**	varchar(10)	N		OPEN or CLOSED.
11	**updated_at**	timestamptz	N	now()	Last update.

Table 26: *gps_stop constraints*

Constraint	Kind	Definition and purpose
gps_stop_pkey	PK	(id)
gps_stop_one_stop_per_device_per_start	UK	(imei, start_recorded_at)
gps_stop_status_values	CK	status IN ('OPEN','CLOSED')
gps_stop_ends_after_it_starts	CK	end_recorded_at >= start_recorded_at

Table 27: *gps_stop indexes*

Index	Columns	Serves
gps_stop_vehicle_time	(vehicle_registration_no, start_recorded_at DESC)	Stops of a vehicle in a period

4.9 Table gps_trip_late_reading

Item	Value
Purpose	Every late reading the trip detector skipped, once; also the work list of the future trip repair job.
Created in	V4
Written by / read by	gps-trip-service / (future) repair job

Table 28: *gps_trip_late_reading columns*

#	Column	Type	Null	Default	Description
1	**imei**	varchar(50)	N		Device.
2	**recorded_at**	timestamptz	N		Time of the late reading.
3	**trip_start_recorded_at**	timestamptz	Y		Start of the trip it fell into (logical reference to gps_trip); null if it fell into a stop or no trip.
4	**noted_at**	timestamptz	N	now()	When it was noted.

Table 29: *gps_trip_late_reading constraints*

Constraint	Kind	Definition and purpose
gps_trip_late_reading_pkey	PK	(imei, recorded_at): a Kafka redelivery is not counted again.

4.10 Table vehicle_event

Item	Value
Purpose	Events raised by the stream consumers (over-speed, sensor mismatch, stoppage, parking, idling, ignition, harsh driving).
Created in	V5
Written by / read by	Speed-violation, vehicle-state, stoppage and ignition consumers / GET /api/events, report summary job, console Alerts
Write pattern	Upsert on event_key: written OPEN when an episode starts and again CLOSED when it ends.

Table 30: *vehicle_event columns*

#	Column	Type	Null	Default	Description
1	**id**	bigint identity	N	generated	Surrogate key.
2	**event_key**	varchar(120)	N		Natural key of the episode (type, device, start).
3	**imei**	varchar(50)	N		Device.
4	**tenant_code**	varchar(40)	Y		Tenant.
5	**vehicle_registration_no**	varchar(50)	Y		Vehicle.
6	**event_type**	varchar(40)	N		OVERSPEED, SPEED_SENSOR_MISMATCH, STOPPAGE, PARKED, IDLING, IGNITION_ON, IGNITION_OFF, HARSH_ACCELERATION, HARSH_BRAKING.
7	**severity**	varchar(10)	N		INFO, LOW, MEDIUM, HIGH, CRITICAL (not checked by the database).
8	**status**	varchar(10)	N		OPEN or CLOSED.
9	**started_at**	timestamptz	N		Episode start.
10	**ended_at**	timestamptz	Y		Episode end; null while OPEN.
11	**latitude / longitude**	double precision	N		Where the episode started.
12	**detected_by**	varchar(40)	N		Consumer that raised it.
13	**details**	jsonb	N	'{}'	Type-specific facts (limit, peak speed, verdict, agents' findings …).
14	**created_at**	timestamptz	N	now()	First write.
15	**updated_at**	timestamptz	N	now()	Last write.

Table 31: *vehicle_event constraints*

Constraint	Kind	Definition and purpose
vehicle_event_pkey	PK	(id)
vehicle_event_event_key_key	UK	(event_key): redelivery and the CLOSED update hit the same row.
vehicle_event_status_values	CK	status IN ('OPEN','CLOSED')

Table 32: *vehicle_event indexes*

Index	Columns	Serves
vehicle_event_type_time	(event_type, started_at DESC)	Events of a type
vehicle_event_imei_time	(imei, started_at DESC)	Events of a vehicle
vehicle_event_recent	(updated_at DESC)	Newest first (GET /api/events)

4.11 Table dead_letter

Item	Value
Purpose	Permanent archive of every record a consumer or Kafka Streams application could not handle.
Created in	V6
Written by / read by	gps-dead-letter-archive / Storage → Dead letters, GET /api/dead-letters
Write pattern	Insert once per dead letter; afterwards only status, replay_count, last_replayed_at and resolution_note change. Never deleted (except demo reset).

Table 33: *dead_letter columns*

#	Column	Type	Null	Default	Description
1	**id**	bigint identity	N	generated	Surrogate key.
2	**source**	varchar(20)	N		Listener or Kafka Streams application that parked it.
3	**original_topic**	varchar(250)	Y		Topic the record came from.
4	**original_partition**	integer	Y		Its partition.
5	**original_offset**	bigint	Y		Its offset.
6	**consumer_group**	varchar(250)	Y		Group that failed.
7	**record_key**	varchar(250)	Y		Record key (IMEI).
8	**record_value**	text	Y		Record value exactly as received.
9	**exception_class**	varchar(500)	Y		Error type.
10	**exception_message**	text	Y		Error message.
11	**stacktrace**	text	Y		Stack trace.
12	**parked_at**	timestamptz	N		When it was parked.
13	**dead_letter_partition**	integer	N		Partition in gps.dead-letters.
14	**dead_letter_offset**	bigint	N		Offset in gps.dead-letters.
15	**status**	varchar(10)	N	'PARKED'	PARKED, REPLAYED or DISCARDED.
16	**replay_count**	integer	N	0	Times replayed.
17	**last_replayed_at**	timestamptz	Y		Last replay.
18	**resolution_note**	text	Y		Why it was discarded (required for DISCARDED).
19	**archived_at**	timestamptz	N	now()	When the row was written.

Table 34: *dead_letter constraints*

Constraint	Kind	Definition and purpose
dead_letter_pkey	PK	(id)
dead_letter_status_values	CK	status IN ('PARKED','REPLAYED','DISCARDED')
dead_letter_archived_once	UK	(dead_letter_partition, dead_letter_offset, parked_at): archived once, even if the topic is recreated.

Table 35: *dead_letter indexes*

Index	Columns	Serves
dead_letter_status_time	(status, parked_at DESC)	Parked letters, newest first

Index	Columns	Serves
dead_letter_topic_time	(original_topic, parked_at DESC)	Replay the oldest parked of a topic
dead_letter_key	(record_key)	Dead letters of a device

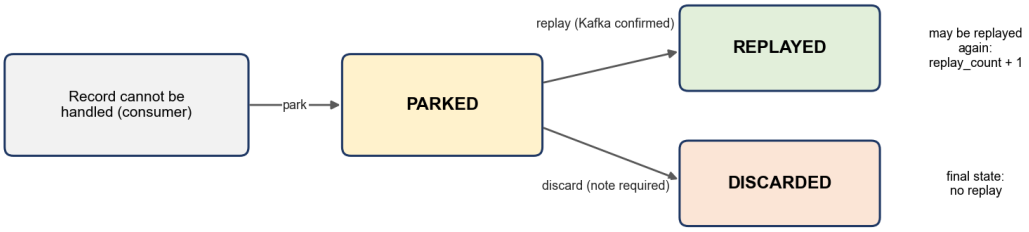


Figure 2: *Status life cycle of dead_letter*

4.12 Table rejected_frame

Item	Value
Purpose	Permanent archive of frames the normaliser refused, with the reason and the payload exactly as received.
Created in	V6
Written by / read by	gps-rejected-frame-logger / Storage → Rejected frames, GET /api/rejected-frames

Table 36: *rejected_frame columns*

#	Column	Type	Null	Default	Description
1	**id**	bigint identity	N	generated	Surrogate key.
2	**imei**	varchar(50)	Y		IMEI if it could be read.
3	**rejection_reason**	text	N		Reason (for example “Latitude 95.0 is outside -90..90”).
4	**original_payload**	text	N		Frame as received.
5	**rejected_at**	timestamptz	N		Time of rejection.
6	**payload_hash**	char(32)	N		Hash of the payload, for de-duplication.
7	**archived_at**	timestamptz	N	now()	When the row was written.

Table 37: *rejected_frame constraints*

Constraint	Kind	Definition and purpose
rejected_frame_pkey	PK	(id)
rejected_frame_archived_once	UK	(rejected_at, payload_hash): a redelivered rejection is stored once.

Table 38: *rejected_frame indexes*

Index	Columns	Serves
rejected_frame_time	(rejected_at DESC)	Newest rejections
rejected_frame_imei_time	(imei, rejected_at DESC)	Rejections of a device

4.13 Table connection_setting

Item	Value
Purpose	External addresses changed from the console while the service runs; a missing row means the application.yml default.
Created in	V7
Why in PostgreSQL	The Redis address cannot be kept in the Redis it points to.

Table 39: connection_setting columns

#	Column	Type	Null	Default	Description
1	**name**	varchar(40)	N		redis or valhalla.
2	**url**	varchar(500)	N		redis://host:port/database or http://host:port.
3	**updated_at**	timestamptz	N	now()	Last change.

Table 40: connection_setting constraints

Constraint	Kind	Definition and purpose
connection_setting_pkey	PK	(name)

4.14 Table vehicle_day_summary

Item	Value
Purpose	One row per vehicle per day (and generation): the source of all day and month reports.
Created in	V8
Written by / read by	DaySummaryJob (today and yesterday every 5 minutes; backfill at start; on demand) / GET /api/reports/*
Generations	0 = live row, recomputed from the raw tables. Clear database freezes the live rows into a new generation > 0 before emptying the raw tables; a report adds all generations of a day.
Expected size	About 150 rows a day (4,650 a month) for 150 vehicles.

Table 41: vehicle_day_summary columns

#	Column	Type	Null	Default	Description
1	**day**	date	N		Report day (Asia/Kolkata).
2	**imei**	varchar(50)	N		Device.
3	**generation**	integer	N	0	0 live; > 0 frozen by Clear database.
4	**tenant_code**	varchar(40)	Y		Tenant.
5	**vehicle_registration_no**	varchar(50)	Y		Vehicle.
6	**readings**	integer	N		Positions in the day.
7	**first_seen_at / last_seen_at**	timestamptz	Y		First and last reading of the day.
8	**distance_m**	double precision	N		Distance driven.
9	**moving_s / stopped_s**	bigint	N		Driving and standing time (long silences excluded).
10	**max_speed_kmph**	integer	N		Highest speed.

#	Column	Type	Null	Default	Description
11	**fast_distance_m**	double precision	N		Distance driven above economical_speed_kmph.
12	**economical_speed_kmph**	integer	N		Economical speed in force when computed.
13	**idle_s**	bigint	N		Time in IDLING episodes.
14	**stops / stop_s**	integer / bigint	N		Number and total time of stops.
15	**engine_on_stops**	integer	N		Stops with the engine running.
16	**trips**	integer	N		Trips started in the day.
17	**overspeed_events / harsh_events**	integer	N		Event counts.
18	**computed_at**	timestamptz	N	now()	When computed.

Table 42: *vehicle_day_summary constraints*

Constraint	Kind	Definition and purpose
vehicle_day_summary_pkey	PK	(day, imei, generation)

Table 43: *vehicle_day_summary indexes*

Index	Columns	Serves
vehicle_day_summary_vehicle	(imei, day)	Month of one vehicle

4.15 Table report_setting

Item	Value
Purpose	Cost assumptions edited on Reports → Cost assumptions; a missing row means the built-in default. Not touched by Clear database.
Created in	V8

Table 44: *report_setting columns*

#	Column	Type	Null	Default	Description
1	**name**	varchar(60)	N		Setting name (Table below).
2	**value**	varchar(100)	N		Value as text.
3	**updated_at**	timestamptz	N	now()	Last change.

Table 45: *report_setting constraints*

Constraint	Kind	Definition and purpose
report_setting_pkey	PK	(name)

Table 46: *report_setting names and defaults*

name	Default	Validation
fuel-type	diesel	diesel or petrol
price-per-litre	92.0	≥ 0 (rupees)
mileage-km-per-litre	4.0	> 0

name	Default	Validation
idle-litres-per-hour	2.5	≥ 0
economical-speed-kmph	60	10 to 150; a change rebuilds stored days
fast-driving-penalty-percent	20.0	≥ 0
co2-kg-per-litre	2.68 (petrol 2.31)	≥ 0

5 Relationships and Referential Integrity

Table 47: *Relationships*

Parent	Child	Cardinality	Enforced by	Rule
tenant	vehicle	0..1 : N	FK	A vehicle may have no owner (market vehicle).
device	device_assignment	1 : N	FK + EX	Periods of one device never overlap.
vehicle	device_assignment	1 : N	FK	A vehicle may carry several devices (GPS, fuel …).
device (imei)	gps_position, vehicle_motion_state, gps_trip, gps_stop, gps_trip_late_reading, vehicle_event, vehicle_day_summary, rejected_frame	1 : N	Logical	Rows are kept even if the device is retired.
tenant (code)	telemetry tables (tenant_code)	1 : N	Logical	Code is never reused, so history stays unambiguous.
vehicle (registration_no)	telemetry tables (vehicle_registration_no)	1 : N	Logical	Copied at the reading's time (BR-004).
device_assignment	vehicle_motion_state.assignment_id	1 : N	Logical	A change of assignment closes the open trip.
gps_trip	gps_trip_late_reading.trip_start_recorded_at	1 : N	Logical (imei + start)	Late readings of a trip.

Logical references are deliberate (SRS DR-002). The application guarantees them as follows: tenant and vehicle are resolved from device_assignment for every reading (or left null with a WARN log); codes and registration numbers are never reused; master-data rows are never deleted while telemetry refers to them.

6 Value Sets

Table 48: *Enumerated values*

Column(s)	Allowed values	Checked by
tenant.status	ACTIVE, SUSPENDED, CLOSED	CK
device.kind	GPS, FUEL, TEMPERATURE, DOOR, OBD	CK
gps_position.movement_status	MOVING, STOPPED	CK
gps_position.distance_source	FIRST_READING, GPS, DEVICE_ODOMETER, JITTER_FILTERED, JUMP_FILTERED, OUT_OF_ORDER	Application (enum)
gps_trip.status, gps_stop.status, vehicle_event.status	OPEN, CLOSED	CK
vehicle_event.event_type	OVERSPEED, SPEED_SENSOR_MISMATCH, STOPPAGE, PARKED, IDLING, IGNITION_ON, IGNITION_OFF, HARSH_ACCELERATION, HARSH_BRAKING	Application (enum)
vehicle_event.severity	INFO, LOW, MEDIUM, HIGH, CRITICAL	Application (enum)
dead_letter.status	PARKED, REPLAYED, DISCARDED	CK
connection_setting.name	redis, valhalla	Application

Note: event_type, severity and distance_source are checked only by the application. Adding CHECK constraints for them is recommended in a future migration (open issue DB-OI-02).

7 Index Strategy and Main Queries

Every frequent query is served by an index whose leading columns match its filter. The main access paths are:

Table 49: *Main queries and the index that serves them*

Query	Index	Frequency
Duplicate check before insert: exists (imei, recorded_at)	unique (imei, recorded_at)	Every position
Previous reading after restart: newest row of an imei	unique (imei, recorded_at)	First reading per device after start
Latest position of a vehicle	gps_position_vehicle_time	API, on demand
One day of positions of all vehicles (summary job)	gps_position_recorded_at	Every 5 minutes, two days
Trips / stops of a vehicle overlapping a period	gps_trip_vehicle_time, gps_stop_vehicle_time	API, reports
Newest events, events of a type or a vehicle	vehicle_event_recent, _type_time, _imei_time	Console polling (2 s)
Parked dead letters; oldest parked of a topic	dead_letter_status_time, dead_letter_topic_time	Console, replay
Month of one vehicle	vehicle_day_summary_vehicle	Reports

```
-- Assignment valid at a reading's time (served by the exclusion-constraint gist index)
SELECT a.id, v.registration_no, t.code
  FROM device_assignment a
  JOIN device  d ON d.id = a.device_id
  JOIN vehicle v ON v.id = a.vehicle_id
  LEFT JOIN tenant t ON t.id = v.owner_tenant_id
 WHERE d.imei = :imei
   AND a.valid_from <= :recordedAt AND (a.valid_to IS NULL OR a.valid_to > :recordedAt);
```

8 Data Integrity, Idempotency and Concurrency

Table 50: *Idempotency key of each consumer-written table*

Table	Key	On redelivery
gps_position	(imei, recorded_at)	Existence check first; unique key as the last line of defence
gps_trip, gps_stop	(imei, start_recorded_at)	Upsert updates the same row
vehicle_motion_state	imei + guard last_recorded_at < :recordedAt	0 rows updated → transaction rolled back
gps_trip_late_reading	(imei, recorded_at)	Insert … ON CONFLICT DO NOTHING; late_reading_count not raised twice
vehicle_event	event_key	Upsert (OPEN then CLOSED)
dead_letter	(dead_letter_partition, dead_letter_offset, parked_at)	Stored once
rejected_frame	(rejected_at, payload_hash)	Stored once
vehicle_day_summary	(day, imei, generation)	Generation 0 replaced on every rebuild

Guarded update used by the trip detector (one transaction per reading, together with the trip and stop upserts):

```
UPDATE vehicle_motion_state
   SET motion_streak = :motionStreak, event_recorded_at = :eventAt, event_latitude = :eventLat,
       event_longitude = :eventLon, last_recorded_at = :recordedAt, reading_window = :window::jsonb,
       open_trip = :openTrip::jsonb, open_stop = :openStop::jsonb, assignment_id = :assignmentId,
       tenant_code = :tenant, vehicle_registration_no = :vehicle, updated_at = now()
 WHERE imei = :imei
   AND last_recorded_at < :recordedAt;   -- 0 rows: a later or equal reading was already applied
```

Concurrency: all readings of one device are in one Kafka partition and therefore handled by one thread at a time, so rows keyed by IMEI have a single writer. Configuration rows (report_setting, connection_setting) are last-writer-wins.

9 Data Lifecycle: Retention, Clearing, Backup and Recovery

9.1 Retention

Table 51: Retention by data store

Data	Retention	Basis
Kafka topics	31 days, by age only	gps.kafka.retention; SRS NFR-CAP-002
Telemetry and trip tables	Until deleted (no automatic deletion yet)	Retention policy is open: SRS NFR-PRV-003, OI-04
dead_letter, rejected_frame	Indefinite	Never deleted except by demo reset (SRS FR-DLQ-008)
vehicle_day_summary	Indefinite	Small; frozen generations survive Clear database
Master data, settings	Indefinite	Survive Clear database and demo reset
Redis live state	Until replaced; history about 1000 entries per vehicle	Rebuilt from the next readings

9.2 Clear Database and Demo Reset

Table 52: What is emptied

Table	Clear database (keep reports)	Demo reset / clear without reports
gps_position, gps_trip, gps_stop, gps_trip_late_reading, vehicle_motion_state, vehicle_event	Emptied (TRUNCATE … RESTART IDENTITY)	Emptied
dead_letter, rejected_frame	Emptied	Emptied
vehicle_day_summary	Kept; live rows frozen into a new generation first	Emptied
tenant, vehicle, device, device_assignment	Kept	Kept
report_setting, connection_setting	Kept	Kept

9.3 Backup and Recovery

- Proposed (SRS DR-008): a daily logical backup (pg_dump -Fc gps_tracking), kept 31 days, with a quarterly restore test.
- Recovery to a point in time within 31 days: restore the newest backup older than the target, then reset the consumer groups' offsets to the backup time and let them re-consume Kafka. Idempotency keys (Section 8) make the overlap harmless.
- Redis needs no backup: live state is rebuilt from the next readings; speed rules (gps:config:*) should be exported with the database backup.

9.4 Capacity Estimate

Table 53: Growth estimate for 150 vehicles

Table	Rows per day	Approx. size per day (with indexes)
gps_position (1 reading/min)	216,000	about 60 to 80 MB

Table	Rows per day	Approx. size per day (with indexes)
gps_position (compressed simulator)	about 6,500,000	about 2 GB; demo use only
gps_trip + gps_stop	a few thousand	under 2 MB
vehicle_event	a few hundred to thousands	under 2 MB
vehicle_day_summary	150	negligible

Note: Sizes are estimates (about 250 to 350 bytes per gps_position row including indexes) and shall be confirmed with pg_total_relation_size after a week of normal operation.

10 Migration History and Change Rules

Table 54: *Flyway migrations*

Version	File	Change
V1	V1__create_master_data_tables.sql	btree_gist; tenant, vehicle, device, device_assignment
V2	V2__create_gps_position_table.sql	gps_position with checks, unique key and indexes
V3	V3__create_vehicle_motion_state_table.sql	Trip-detector state
V4	V4__create_gps_trip_and_gps_stop_tables.sql	gps_trip, gps_stop, gps_trip_late_reading
V5	V5__distance_details_and_vehicle_events.sql	gps_position distance columns, ignition, odometer; vehicle_event
V6	V6__dead_letter_and_rejected_frame_archive.sql	dead_letter, rejected_frame
V7	V7__connection_setting.sql	connection_setting
V8	V8__vehicle_day_summary_and_report_setting.sql	vehicle_day_summary, report_setting, index gps_position_recorded_at

- An applied migration shall never be edited: Flyway's checksum check stops the service. Every change is a new file V9__<description>.sql, V10__ …
- Every new column or table shall be added to the JPA entities (where one exists) in the same change, because Hibernate validates the schema at start.
- A migration that rewrites a large table (for example adding a NOT NULL column with a default to gps_position) shall be reviewed for lock time and run outside peak hours.
- This document shall be updated in the same change as the migration (SRS Rules and Regulations, change control).

11 Non-Relational Data Stores

11.1 Redis Key Space

Table 55: *Redis keys*

Key pattern	Type	Written by	Content / lifetime
gps:live:vehicle:{imei}	hash	live-location	vehicle, tenant, lat, lon, speed, status, recordedAt, updatedAt; newer wins
gps:live:positions	geo set	live-location	One member per IMEI, for GEOSEARCH by radius
gps:state:vehicle:{imei}	hash	vehicle-state	Heading, acceleration, Kalman track, counters
gps:state:speed:{imei}	hash	speed-violation	Current episode, GPS window, verdict
gps:state:ignition:{imei}	hash	ignition	Ignition state, engine hours, idling
gps:state:stoppage:{imei}	hash	stoppage	Stay-point candidate or current stop
gps:history:{imei}	stream	vehicle-state	XADD per reading, XTRIM MAXLEN ~1000
gps:events:{imei}	list	event recorder	Recent events of the vehicle
gps:context:weather:{cell}	string	stoppage agents	Weather of a 0.1° cell, 10-minute expiry
gps:config:speed-rules	set	speed rules API	Scopes that have a rule
gps:config:speed-rule:{scope}	hash	speed rules API	Rule fields (Section 3.9 of the SRS); kept by resets
gps:engine:stage:{stage}	hash	metrics mirror	Totals, rate, failures, last message; every 5 s
gps:engine:stage:{stage}:by-imei	hash	metrics mirror	Messages per IMEI for the stage

Telemetry patterns deleted by the demo reset: gps:live:*, gps:engine:*, gps:state:*, gps:history:*, gps:events:*, gps:context:*. Configuration (gps:config:*) and other programs' keys are never deleted.

11.2 Kafka Topics

Table 56: *Topics (all keyed by IMEI, 3 partitions, 31-day retention, zstd)*

Topic	Value	Notes
gps.raw-device-frames	Raw frame JSON	Written through the ingest journal
gps.processed-positions	Processed position JSON	Transactional (exactly-once); timestamp = recordedAt
gps.rejected-frames	Rejected frame JSON	Transactional
gps.vehicle-events	Vehicle event JSON	Written through the ingest journal
gps.vehicle-activity	Activity window JSON	One final record per closed 5-minute window
gps.dead-letters	Original bytes + error headers	Archived to dead_letter
gps.shadow.processed-positions / rejected-frames	As above	Shadow mode only

11.3 Kafka Streams State Stores

Table 57: *State stores (RocksDB, each backed by a compacted changelog topic)*

Store	Application	Content
previous-reading-store	normaliser	Newest reading per IMEI (distance and order)
seen-reading-store	normaliser	Window store of readings seen in the last 24 h (duplicates)
vehicle-table-store	analytics	KTable: latest snapshot per vehicle
activity-window-store	analytics	5-minute windows, kept 6 h for queries

12 External Source: smart_truck (Read-Only)

The simulator reads the legacy MySQL database smart_truck through a separate read-only pool, with SELECT statements only (SRS CON-002). Times in smart_truck are India time and are converted to UTC instants.

Table 58: *smart_truck columns read*

Table.column	Meaning	Becomes
tta_trips.i_trip_no	Trip number	Selects the latest trip per device
tta_trips.i_gps_ping_count	Pings in the trip	Filter: at least 200 (minimum-pings-per-trip)
tta_trip_gps.s_device_id	Device identifier	imei
tta_trip_gps.s_asset_id	Vehicle identifier	vehicle.registration_no
tta_trip_gps.s_entity_name	Transporter name	tenant.legal_name → tenant.code
tta_trip_gps.dt_message	Ping time (IST)	recordedAt (UTC)
tta_trip_gps.d_lat / d_long	Position	latitude / longitude
tta_trip_gps.i_speed	Speed km/h	speedKmph
tta_trip_gps.i_dist	Distance increment	Simulated odometer (for 50 % of devices)

13 Open Issues

Table 59: *Database open issues*

ID	Issue	Owner
DB-OI-01	Define and implement retention and deletion for telemetry tables (time-based partitions on gps_position are the likely solution).	Technical lead
DB-OI-02	Add CHECK constraints for vehicle_event.event_type and severity and gps_position.distance_source.	Developer
DB-OI-03	Decide the time-series store (ClickHouse is planned) and the hand-over of gps_position history.	Architect
DB-OI-04	Master-data administration API: who may create, change and close tenants, vehicles, devices and assignments.	Product owner
DB-OI-05	Automated backup and restore test (SRS DR-008).	Operations

14 Appendices

14.1 Appendix A: Complete DDL

The exact text of the Flyway migrations, in order. This appendix is regenerated from the files whenever the document is built.

14.1.1 V1__create_master_data_tables.sql

```
-- Master data needed to turn a device IMEI into a vehicle and a tenant.
-- This is the first slice of the GPS master-data design: tenant, vehicle, device, device_assignment.
-- The remaining tables (tenant_role, branch, app_user, trip, shipment, tracking_grant ...) follow in
later migrations.

-- Needed for the "one vehicle per device at a time" exclusion constraint below.
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE tenant (
    id          uuid         PRIMARY KEY,
    code        varchar(40)  NOT NULL UNIQUE,
    legal_name  varchar(255) NOT NULL,
    status      varchar(20)  NOT NULL DEFAULT 'ACTIVE',
    created_at  timestamptz  NOT NULL DEFAULT now(),
    -- code is stamped on every GPS position, so it is short, lower-case and never re-used
    CONSTRAINT tenant_code_format CHECK (code ~ '^[a-z0-9][a-z0-9_-]*$'),
    CONSTRAINT tenant_status_values CHECK (status IN ('ACTIVE', 'SUSPENDED', 'CLOSED'))
);

CREATE TABLE vehicle (
    id              uuid        PRIMARY KEY,
    -- one truck is one truck: the plate is unique across the whole platform
    registration_no  varchar(50) NOT NULL UNIQUE,
    -- null = market vehicle (not owned by a tenant on the platform)
    owner_tenant_id  uuid        REFERENCES tenant (id),
    created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE device (
    id          uuid        PRIMARY KEY,
    imei        varchar(50) NOT NULL UNIQUE,
    kind        varchar(20) NOT NULL DEFAULT 'GPS',
    protocol    varchar(50),
    created_at  timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT device_kind_values CHECK (kind IN ('GPS', 'FUEL', 'TEMPERATURE', 'DOOR', 'OBD'))
);

-- Stored as periods, never as "current vehicle": a reading delivered late must be matched
-- to the vehicle the device was in at the time of the reading.
CREATE TABLE device_assignment (
    id          uuid        PRIMARY KEY,
    device_id   uuid        NOT NULL REFERENCES device (id),
    vehicle_id  uuid        NOT NULL REFERENCES vehicle (id),
    valid_from  timestamptz NOT NULL,
    valid_to    timestamptz,
    CONSTRAINT device_assignment_period_is_valid CHECK (valid_to IS NULL OR valid_to > valid_from),
    -- A device can be in only one vehicle at a time, so every reading resolves to exactly one truck.
    CONSTRAINT device_assignment_one_vehicle_per_device_at_a_time
        EXCLUDE USING gist (device_id WITH =, tstzrange(valid_from, valid_to) WITH &&)
);

CREATE INDEX device_assignment_vehicle ON device_assignment (vehicle_id);
```

14.1.2 V2__create_gps_position_table.sql

```
-- GPS positions after Flink processing, with the tenant and vehicle resolved from master data.
-- Tenant and vehicle are stored as readable codes (not foreign keys): telemetry is written fast,
-- read in reports, and will later move to a time-series store.

CREATE TABLE gps_position (
    id                      bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    imei                    varchar(50)      NOT NULL,
    tenant_code             varchar(40),
    vehicle_registration_no  varchar(50),
```

```
    recorded_at               timestamptz      NOT NULL,
    latitude                  double precision NOT NULL,
    longitude                 double precision NOT NULL,
    speed_kmph                integer          NOT NULL,
    movement_status           varchar(10)      NOT NULL,
    distance_from_previous_m  double precision NOT NULL,
    seconds_since_previous    bigint           NOT NULL,
    arrived_out_of_order      boolean          NOT NULL,
    processed_at              timestamptz      NOT NULL,
    stored_at                 timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT gps_position_latitude_range  CHECK (latitude BETWEEN -90 AND 90),
    CONSTRAINT gps_position_longitude_range CHECK (longitude BETWEEN -180 AND 180),
    CONSTRAINT gps_position_movement_status_values CHECK (movement_status IN ('MOVING', 'STOPPED')),
    -- Kafka delivers at-least-once, so the same reading can arrive twice; it is stored once.
    CONSTRAINT gps_position_one_reading_per_device_per_moment UNIQUE (imei, recorded_at)
);

CREATE INDEX gps_position_vehicle_time ON gps_position (vehicle_registration_no, recorded_at DESC);
CREATE INDEX gps_position_stored_at    ON gps_position (stored_at DESC);
```

14.1.3 V3__create_vehicle_motion_state_table.sql

```
-- State of the trip detection (consumer group gps-trip-service): one row per device.
-- Traccar keeps the same on tc_devices (motionstreak, motiontime, motionlatitude, motionlongitude)
-- plus the recent positions in memory. Here it is one row, updated in the SAME transaction as the
-- gps_trip / gps_stop rows it produces, so a trip can never be written twice or lost half-way.
--
-- Every update is guarded by "last_recorded_at < the new reading's time": a reading that is late,
-- or delivered again by Kafka, updates 0 rows and changes nothing.

CREATE TABLE vehicle_motion_state (
    imei                    varchar(50)      PRIMARY KEY,
    -- the device_assignment of the newest reading; a change closes the open trip (null: not in a
vehicle)
    assignment_id           uuid,
    tenant_code             varchar(40),
    vehicle_registration_no  varchar(50),
    -- true between a "started moving" and a "stopped" decision (Traccar's motionStreak)
    motion_streak           boolean          NOT NULL,
    -- where and when the vehicle last started moving or stopped
    event_recorded_at       timestamptz      NOT NULL,
    event_latitude          double precision NOT NULL,
    event_longitude         double precision NOT NULL,
    -- the newest reading applied: older or equal readings are skipped
    last_recorded_at        timestamptz      NOT NULL,
    -- in-order readings of the last minDuration (a few dozen small objects), oldest first.
    -- Named reading_window because WINDOW is a reserved word in PostgreSQL.
    reading_window          jsonb            NOT NULL,
    -- the trip or stop going on now (the same values as its OPEN row in gps_trip / gps_stop), or null
    open_trip               jsonb,
    open_stop               jsonb,
    updated_at              timestamptz      NOT NULL DEFAULT now()
);
```

14.1.4 V4__create_gps_trip_and_gps_stop_tables.sql

```
-- Trips and stops detected by consumer group gps-trip-service (Traccar computes these on request;
-- here they are written as the readings arrive). Tenant and vehicle are readable codes, as in
gps_position,
-- taken from the device_assignment valid at the time of the readings.
--
-- A trip or stop is identified by (imei, start_recorded_at): a replayed reading produces the same
key,
-- and the row is updated instead of stored twice.

CREATE TABLE gps_trip (
    id                       bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    imei                     varchar(50)      NOT NULL,
    tenant_code              varchar(40),
    vehicle_registration_no  varchar(50),
    start_recorded_at        timestamptz      NOT NULL,
    end_recorded_at          timestamptz      NOT NULL,
    start_latitude           double precision NOT NULL,
    start_longitude          double precision NOT NULL,
    end_latitude             double precision NOT NULL,
    end_longitude            double precision NOT NULL,
```

```
    -- sum of distance_from_previous of the in-order readings, never "end odometer - start odometer"
    distance_m                double precision NOT NULL,
    duration_s                bigint           NOT NULL,
    max_speed_kmph            integer          NOT NULL,
    avg_speed_kmph            double precision NOT NULL,
    reading_count             integer          NOT NULL,
    -- readings that arrived late for this trip; they are not in distance/duration (see
gps_trip_late_reading)
    late_reading_count        integer          NOT NULL DEFAULT 0,
    -- OPEN: the vehicle is still driving, the end moves with every reading. CLOSED: final.
    status                    varchar(10)      NOT NULL,
    updated_at                timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT gps_trip_status_values CHECK (status IN ('OPEN', 'CLOSED')),
    CONSTRAINT gps_trip_ends_after_it_starts CHECK (end_recorded_at >= start_recorded_at),
    -- Traccar's trips report once said -1020 m because of one late reading; this table refuses that.
    CONSTRAINT gps_trip_distance_not_negative CHECK (distance_m >= 0),
    CONSTRAINT gps_trip_one_trip_per_device_per_start UNIQUE (imei, start_recorded_at)
);

CREATE INDEX gps_trip_vehicle_time ON gps_trip (vehicle_registration_no, start_recorded_at DESC);

CREATE TABLE gps_stop (
    id                        bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    imei                      varchar(50)      NOT NULL,
    tenant_code               varchar(40),
    vehicle_registration_no   varchar(50),
    start_recorded_at         timestamptz      NOT NULL,
    end_recorded_at           timestamptz      NOT NULL,
    latitude                  double precision NOT NULL,
    longitude                 double precision NOT NULL,
    duration_s                bigint           NOT NULL,
    reading_count             integer          NOT NULL,
    status                    varchar(10)      NOT NULL,
    updated_at                timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT gps_stop_status_values CHECK (status IN ('OPEN', 'CLOSED')),
    CONSTRAINT gps_stop_ends_after_it_starts CHECK (end_recorded_at >= start_recorded_at),
    CONSTRAINT gps_stop_one_stop_per_device_per_start UNIQUE (imei, start_recorded_at)
);

CREATE INDEX gps_stop_vehicle_time ON gps_stop (vehicle_registration_no, start_recorded_at DESC);

-- Every late reading the trip detection skipped, once (a Kafka redelivery is not counted again).
-- trip_start_recorded_at points to the trip it fell into (null: it fell into a stop or no trip).
-- This is also the work list for a later "repair" job that recomputes those trips from gps_position.
CREATE TABLE gps_trip_late_reading (
    imei                      varchar(50)  NOT NULL,
    recorded_at               timestamptz  NOT NULL,
    trip_start_recorded_at    timestamptz,
    noted_at                  timestamptz  NOT NULL DEFAULT now(),
    PRIMARY KEY (imei, recorded_at)
);
```

14.1.5 V5__distance_details_and_vehicle_events.sql

```
-- Distance as Traccar's DistanceHandler computes it (see CalculateMovementStep), and what the device
reported.
ALTER TABLE gps_position
    ADD COLUMN total_distance_m double precision,
    ADD COLUMN distance_source  varchar(20),
    ADD COLUMN ignition         boolean,
    ADD COLUMN odometer_m       bigint;

-- Events raised by the stream consumers (over-speed, stoppage, parking, idling, ignition, harsh
driving ...).
-- An episode is written when it starts (OPEN) and again when it ends (CLOSED) with the same
event_key,
-- so the row is upserted - which also makes a Kafka redelivery harmless.
CREATE TABLE vehicle_event (
    id                        bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    event_key                 varchar(120) NOT NULL UNIQUE,
    imei                      varchar(50)  NOT NULL,
    tenant_code               varchar(40),
    vehicle_registration_no   varchar(50),
    event_type                varchar(40)  NOT NULL,
    severity                  varchar(10)  NOT NULL,
    status                    varchar(10)  NOT NULL,
    started_at                timestamptz  NOT NULL,
```

```
    ended_at                timestamptz,
    latitude                double precision NOT NULL,
    longitude               double precision NOT NULL,
    detected_by             varchar(40)  NOT NULL,
    details                 jsonb        NOT NULL DEFAULT '{}'::jsonb,
    created_at              timestamptz  NOT NULL DEFAULT now(),
    updated_at              timestamptz  NOT NULL DEFAULT now(),
    CONSTRAINT vehicle_event_status_values CHECK (status IN ('OPEN', 'CLOSED'))
);

CREATE INDEX vehicle_event_type_time   ON vehicle_event (event_type, started_at DESC);
CREATE INDEX vehicle_event_imei_time   ON vehicle_event (imei, started_at DESC);
CREATE INDEX vehicle_event_recent      ON vehicle_event (updated_at DESC);
```

14.1.6 V6__dead_letter_and_rejected_frame_archive.sql

```
-- Nothing the platform receives is ever thrown away. Kafka keeps every topic for a month; these two
tables keep the
-- records that did NOT become positions for good, so they outlive Kafka's retention.

-- Dead letters: records a consumer (listener or Kafka Streams) could not handle and parked in
gps.dead-letters.
-- Rows are never deleted: a replay or a discard only changes the status, so the history of every
record stays.
CREATE TABLE dead_letter (
    id                      bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    source                  varchar(20)  NOT NULL,
    original_topic          varchar(250),
    original_partition      integer,
    original_offset         bigint,
    consumer_group          varchar(250),
    record_key              varchar(250),
    record_value            text,
    exception_class         varchar(500),
    exception_message       text,
    stacktrace              text,
    parked_at               timestamptz  NOT NULL,
    dead_letter_partition   integer      NOT NULL,
    dead_letter_offset      bigint       NOT NULL,
    status                  varchar(10)  NOT NULL DEFAULT 'PARKED',
    replay_count            integer      NOT NULL DEFAULT 0,
    last_replayed_at        timestamptz,
    resolution_note         text,
    archived_at             timestamptz  NOT NULL DEFAULT now(),
    CONSTRAINT dead_letter_status_values CHECK (status IN ('PARKED', 'REPLAYED', 'DISCARDED')),
    -- the same dead letter delivered twice is archived once (offset + time: unique even if the topic
is ever recreated)
    CONSTRAINT dead_letter_archived_once UNIQUE (dead_letter_partition, dead_letter_offset, parked_at)
);

CREATE INDEX dead_letter_status_time ON dead_letter (status, parked_at DESC);
CREATE INDEX dead_letter_topic_time  ON dead_letter (original_topic, parked_at DESC);
CREATE INDEX dead_letter_key         ON dead_letter (record_key);

-- Rejected frames: frames the normalizer refused (bad coordinates, duplicates, stale ...), with the
reason and the
-- payload exactly as it arrived. Until now they only reached the service log.
CREATE TABLE rejected_frame (
    id                bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    imei              varchar(50),
    rejection_reason  text         NOT NULL,
    original_payload  text         NOT NULL,
    rejected_at       timestamptz  NOT NULL,
    payload_hash      char(32)     NOT NULL,
    archived_at       timestamptz  NOT NULL DEFAULT now(),
    -- a redelivered rejection (same payload, same moment) is stored once
    CONSTRAINT rejected_frame_archived_once UNIQUE (rejected_at, payload_hash)
);

CREATE INDEX rejected_frame_time      ON rejected_frame (rejected_at DESC);
CREATE INDEX rejected_frame_imei_time ON rejected_frame (imei, rejected_at DESC);
```

14.1.7 V7__connection_setting.sql

```
-- Addresses of the service's outside connections, changed from the console (Developer -> Connections)
while the
```

```
-- service runs. A missing row means the default from application.yml (gps.connections.*,
spring.data.redis.*).
--   redis     redis://host:port/database   the Redis every consumer keeps its state in (this laptop
or the HP)
--   valhalla  http://host:port             the Valhalla routing engine (all-India map on the HP home
server)
-- Kept in PostgreSQL, not in Redis: the Redis address cannot live in the Redis it points to.
CREATE TABLE connection_setting (
    name        varchar(40)   PRIMARY KEY,
    url         varchar(500)  NOT NULL,
    updated_at  timestamptz   NOT NULL DEFAULT now()
);
```

14.1.8 V8__vehicle_day_summary_and_report_setting.sql

```
-- Reports (Reports section of the console): one row per vehicle per day, so a month for 150 vehicles
is about 4,650
-- small rows instead of millions of positions. Rebuilt from gps_position, gps_stop, gps_trip and
vehicle_event for
-- today and yesterday every few minutes (DaySummaryJob), for older days at start and on demand.
--
-- generation 0 is the live row, recomputed from those tables. "Clear database" first freezes the live
rows into a new
-- generation (> 0) and then empties the tables, so the day's numbers survive the clear; a report adds
the generations
-- of a day up.
CREATE TABLE vehicle_day_summary (
    day                     date              NOT NULL,
    imei                    varchar(50)       NOT NULL,
    generation              integer           NOT NULL DEFAULT 0,
    tenant_code             varchar(40),
    vehicle_registration_no  varchar(50),
    readings                integer           NOT NULL,
    first_seen_at           timestamptz,
    last_seen_at            timestamptz,
    distance_m              double precision  NOT NULL,
    moving_s                bigint            NOT NULL,
    stopped_s               bigint            NOT NULL,
    max_speed_kmph          integer           NOT NULL,
    -- kilometres driven above economical_speed_kmph (the cost assumption in force when the row was
computed)
    fast_distance_m         double precision  NOT NULL,
    economical_speed_kmph   integer           NOT NULL,
    -- IDLING episodes inside the day: standing with the engine on longer than the idling limit
    idle_s                  bigint            NOT NULL,
    stops                   integer           NOT NULL,
    stop_s                  bigint            NOT NULL,
    engine_on_stops         integer           NOT NULL,
    trips                   integer           NOT NULL,
    overspeed_events        integer           NOT NULL,
    harsh_events            integer           NOT NULL,
    computed_at             timestamptz       NOT NULL DEFAULT now(),
    PRIMARY KEY (day, imei, generation)
);

CREATE INDEX vehicle_day_summary_vehicle ON vehicle_day_summary (imei, day);

-- Report assumptions (fuel price, mileage, idle burn ...) edited on Reports -> Cost assumptions. A
missing row means the
-- default in FuelAssumptions.defaults(). Not touched by "Clear database".
CREATE TABLE report_setting (
    name        varchar(60)   PRIMARY KEY,
    value       varchar(100)  NOT NULL,
    updated_at  timestamptz   NOT NULL DEFAULT now()
);

-- The summaries read one day of positions for every vehicle at a time.
CREATE INDEX gps_position_recorded_at ON gps_position (recorded_at);
```